eBPF Transforms Filesystem Monitoring and Intelligent Backup Systems

The Linux kernel's eBPF technology has revolutionized filesystem monitoring, delivering **10-20x performance improvements** over traditional methods ACM Digital Library +3↗ while enabling AI-driven intelligent backup and storage systems. New Relic +2↗ Production deployments at Netflix, Google, and major cloud providers demonstrate eBPF can monitor filesystem changes with **sub-1% overhead**, ACM Digital Library↗ capturing detailed change patterns that feed machine learning models achieving **99%+ accuracy** in predicting file access and detecting threats. Groundcover↗Brendan Gregg↗

This convergence of kernel-level observability and artificial intelligence represents a fundamental shift from traditional scanning-based approaches. Rather than periodically walking directory trees to check modification times, modern eBPF-based systems intercept filesystem operations at the VFS layer in real-time, pre-filter events in kernel space, and stream actionable intelligence to ML models that prioritize backups, predict access patterns, and detect anomalies. Aqua Security +3↗ The technology stack exists today in production-ready open-source and commercial solutions, with clear pathways to intelligent, self-optimizing storage systems.

Production-ready open source solutions lead the ecosystem

The eBPF filesystem monitoring landscape features several mature open-source projects deployed at scale. **FSProbe** stands out as a VFS-level filesystem events notifier that works below typical syscall-based solutions, providing path resolution through dentry tree traversal with three different resolution modes optimized for different scenarios. GitHub↗ Unlike opensnoop or traditional syscall tracers, FSProbe intercepts operations at the Virtual File System layer, making it filesystem-agnostic and harder to bypass. GitHub↗ The project has been battle-tested in production environments and powers an improved version used by Datadog's agent. github↗ Performance benchmarks show FSProbe sustaining **15,000+ events per second** with the perf_buffer method using 60-70% less memory than alternative approaches. GitHub↗

Cilium Tetragon represents the enterprise-grade evolution of eBPF security monitoring, now a CNCF graduated project with Kubernetes-native capabilities. ACM Digital Library↗ Tetragon monitors comprehensive file operations including reads, writes, executions, and memory-mapped file access through Linux Security Module hooks like security_file_permission. Tetragon +4↗ Its declarative YAML-based TracingPolicy system allows administrators to define custom monitoring rules that execute at kernel level with **sub-millisecond latency**. Isovalent↗iximiuz Labs↗ Tetragon uniquely combines observation with runtime enforcement, capable of blocking malicious operations before completion through kernel-level policy execution. Tetragon +2↗ The platform enriches file events with container and Kubernetes metadata, making it ideal for cloud-native environments where traditional monitoring breaks down across namespace boundaries. Isovalent +2↗

The **BCC toolkit** from the IO Visor project provides over 70 pre-built tools forming the foundation of eBPF filesystem analysis. eBPF↗ Tools like vfsstat, filetop, and fileslower offer immediate insights into VFS operations, while filesystem-specific variants (ext4slower, xfsslower, btrfsslower, zfsslower) provide targeted latency analysis. SNIA +4↗ The toolkit's strength lies in its layered approach, with tools spanning syscalls, VFS functions, filesystem-specific operations, and the block layer. Production environments at major tech companies deploy BCC by default across all servers, with overhead low enough for continuous 24x7 operation. Brendan Gregg↗ The biolatency and biosnoop tools trace block device I/O with process correlation, while biopattern identifies whether workloads exhibit random or sequential access characteristics critical for optimization. github +2↗

bpftrace complements BCC with a high-level scripting language inspired by awk and DTrace, enabling custom filesystem monitoring through simple one-liners or complex scripts. eBPF↗ The tool excels at ad-hoc analysis and rapid prototyping, with support for kprobes, tracepoints, and USDT probes. GitHub +3↗ A single bpftrace command can trace all file opens system-wide or create latency histograms for VFS operations in seconds. The built-in aggregation functions (hist, count, sum, avg) enable sophisticated analysis without external post-processing. Linux Foundation↗GitHub↗ With CO-RE support providing "compile once, run everywhere" portability, bpftrace serves both quick investigations and production monitoring.

Commercial platforms emphasize security over backup optimization

The commercial eBPF landscape focuses primarily on security monitoring and observability rather than traditional backup solutions, revealing an underserved market opportunity. **Sysdig** pioneered container-native monitoring in 2013 and contributed 12+ enhancements to the Linux eBPF project, including donating Falco's kernel components to the CNCF. sysdig↗ The platform captures file I/O operations with container-aware namespace context, tracking process-to-file relationships with full metadata enrichment. Sysdig's eBPF instrumentation overhead measures only **marginally greater than kernel modules at less than 5%**, using per-CPU ring buffers for high-throughput data collection. sysdig↗ The dual-product strategy combines Sysdig Monitor for performance troubleshooting with Sysdig Secure for runtime threat detection.

Falco, now a CNCF graduated project, focuses exclusively on behavioral security monitoring at the kernel level. The tool detects suspicious file access to sensitive paths like /etc/shadow and SSH configurations, monitors unexpected writes to system directories, and enriches events with container and Kubernetes metadata. Elastic +5↗ With **24 million Docker Hub downloads** and adoption by over 1,000 organizations including Shopify and Rancher, Falco demonstrates the production maturity of eBPF security monitoring. Sysdig↗ The project integrates with 50+ third-party SIEM and alert systems, establishing itself as the de facto standard for container runtime security. Falco↗ Falco's customizable rule engine allows organizations to define behavioral policies specific to their environments while maintaining near-zero performance overhead through in-kernel filtering. Elastic↗

Aqua Security offers both open-source and commercial solutions, with Tracee providing kernel-level syscall monitoring and Lightning Enforcer delivering behavioral prevention. Lightning Enforcer claims **65% reduction in agent load** versus non-eBPF alternatives while providing drift prevention that detects new binaries dropped to disk. Aqua↗Aqua Security↗ Unlike observation-only tools, Aqua's commercial platform combines detection with prevention capabilities, blocking malicious actions at the kernel level. The company's Team Nautilus threat research team populates pre-built policies, reducing the manual configuration burden. Aqua↗ However, this focus on security rather than data protection illustrates the gap in eBPF-based backup solutions.

Groundcover represents a newer generation of cost-optimized observability platforms built on eBPF foundations. Their proprietary Flora sensor performs filtering and aggregation at the kernel level, tracking filesystem I/O latency and correlating file operations with distributed traces. Groundcover +3↗ The bring-your-own-cloud (BYOC) architecture keeps data in customer VPCs, eliminating data egress costs while maintaining privacy and compliance. Groundcover's flat per-node pricing model contrasts sharply with traditional APM vendors' data-volume-based pricing, with customers reporting **50%+ cost reductions** versus Datadog or New Relic. Network World↗ The platform claims **sub-1% overhead** for comprehensive monitoring with 14x finer temporal resolution than traditional tools like top, demonstrating eBPF's efficiency enables full-fidelity capture without sampling. Groundcover↗

Notably absent from the commercial landscape are mature backup solutions specifically leveraging eBPF for incremental change tracking. While security and observability vendors have embraced the technology, traditional backup vendors like Veeam and Commvault focus on AI-driven scheduling and anomaly detection rather than eBPF-based change detection. This represents a clear market opportunity for intelligent backup systems that combine eBPF's real-time change monitoring with ML-driven prioritization.

Linux VFS architecture offers multiple strategic hooking points

Understanding the Virtual File System architecture reveals optimal attachment points for filesystem monitoring. The VFS layer sits between userspace syscalls and filesystem-specific implementations, providing a unified interface across ext4, XFS, Btrfs, NFS, and other filesystems. SNIA↗Linux Kernel↗ Four primary data structures form the foundation: superblocks representing mounted filesystems, inodes containing file metadata and operations tables, dentries mapping pathname components to inodes through an in-memory cache, and file structures representing open file descriptors with position and access mode information. Star Lab↗

The critical insight for change detection lies in **hooking VFS-layer functions rather than syscalls or filesystem-specific code**. Functions like vfs_write, vfs_writev, and filp_close capture all write operations regardless of which syscall or filesystem implementation handles them. arXiv↗Theshoemaker↗ This approach provides both filesystem-agnostic coverage and protection against userspace manipulation, since VFS functions receive kernel-validated paths rather than potentially malicious user-supplied strings. amandeep singh↗Theshoemaker↗ For comprehensive change tracking, implementations should monitor vfs_write and vfs_writev for content modifications, filp_close for write

completion indicating data persistence, vfs_fsync_range for explicit synchronization, and do_splice_from for splice operations often overlooked by naive implementations.

Metadata changes require additional hooks: **vfs_rename** for file and directory renames, **vfs_unlink** and **do_unlinkat** for deletions, **vfs_mkdir** and **vfs_rmdir** for directory operations, and **vfs_truncate** for file size changes. arXiv↗ Theshoemaker↗ The do_truncate function captures both explicit truncation and open-with-O_TRUNC patterns. Extended attributes and permission changes flow through vfs_setxattr and vfs_setattr, relevant for security monitoring but less critical for backup change detection.

Path resolution from VFS structures presents the primary technical challenge. Functions receive struct file pointers containing a struct dentry and struct vfsmount, requiring dentry tree traversal to construct full paths. arXiv +2↗ Modern kernels provide the bpf_d_path helper for safe path extraction, but this function isn't available at all hook points. The alternative involves manually walking the dentry tree using d_parent pointers until reaching the mount point root, which eBPF's verifier constraints limit through required loop unrolling. github↗Theshoemaker↗ Production implementations like FSProbe employ three strategies: perf_buffer method with userspace path caching proves most memory-efficient, path fragments method builds paths incrementally in kernel, and single_fragment trades memory for simplicity. The perf_buffer approach delivers **60-70% lower memory usage** while sustaining higher event rates. GitHub↗github↗

Fentry and fexit deliver 10x performance gains over kprobes

eBPF attachment method selection fundamentally determines monitoring performance and stability. **Fentry/fexit trampolines**, available in kernel 5.5+ on x86 (6.0+ on ARM), provide the optimal approach for modern systems. These trampolines invoke BPF programs through direct function calls rather than breakpoint exceptions, eliminating the CPU exception handling and context switches that burden traditional kprobes. USENIX↗ Measured overhead shows fentry/fexit operations complete in **less than 50 nanoseconds** with near-zero impact on monitored systems. eunomia↗ USENIX↗ The fexit variant uniquely provides simultaneous access to both function input parameters and return values, enabling efficient success checking without separate instrumentation. Eunomia↗ Performance benchmarks demonstrate **10x speedup** versus kprobes for equivalent functionality. USENIX↗

LSM (Linux Security Module) hooks offer the most stable API for security-focused monitoring, maintained as part of the kernel's security infrastructure. Functions like security_file_permission, security_inode_unlink, and security_mmap_file provide semantically meaningful interception points with access to struct path for simplified path resolution. Tetragon +3↗ The stability advantage proves crucial for production deployments, as LSM hooks rarely change between kernel versions compared to internal VFS functions that may be renamed, inlined, or refactored. Recent kernels also support fmodify_return on whitelisted LSM hooks, enabling BPF programs to override return values and block malicious operations at the kernel level.

Tracepoints represent the fallback choice for broader kernel compatibility, offering stable instrumentation points defined in kernel source as part of the ABI. arXiv↗LinkedIn↗ Tracepoints like syscalls/sys_enter_openat and syscalls/sys_enter_write provide predictable locations with well-defined context structures. Linux Foundation +2↗ The overhead remains low to medium with minimal checking when inactive, making tracepoints production-ready for 24x7 operation. However, tracepoint coverage proves coarser than VFS-level hooks, sometimes missing operations that bypass standard syscall paths. Legacy systems and debugging scenarios may require kprobes despite their **10x higher overhead** and unstable ABI, but modern implementations should prioritize fentry/fexit as the primary choice, LSM hooks for security monitoring, and tracepoints only when others prove unavailable.

Implementation patterns that optimize performance include aggressive in-kernel filtering to reduce userspace transfers, per-CPU maps to avoid lock contention, ring buffers for efficient event delivery with better ordering guarantees than perf buffers, and multi-stage tracking where file opens populate state maps that subsequent close operations query. A production-grade system might mark files as modified in vfs_write, track write counts in per-CPU arrays, confirm persistence in filp_close, send aggregated events via ring buffer, and reset state after backup completion. This pattern reduces event volume by **orders of magnitude** compared to forwarding every write operation to userspace.

Benchmarks show single-digit overhead with dramatic improvements

Quantitative performance data validates eBPF's production readiness for filesystem monitoring. Academic research from Yusheng Zheng et al. measured **userspace eBPF uprobes completing in 314 nanoseconds versus 3,224 nanoseconds for kernel uprobes**, a 10x speedup attributed to eliminating dual context switches. Uretprobes show similar

improvements at 381ns userspace versus 3,996ns kernel. eunomia +2↗ These sub-microsecond latencies enable monitoring at rates traditional tools cannot sustain. The eHashPipe resource monitoring system demonstrates **20% CPU overhead** while achieving 14x finer temporal resolution than the traditional top command with 100% top-k tracking accuracy using only 1.2MB memory overhead. eunomia +2↗

Comparison studies reveal eBPF's dramatic advantages over traditional Linux monitoring mechanisms. A comprehensive analysis found **inotify scales poorly** because it monitors individual inodes separately without recursive watching, exhausting file descriptors in production scenarios with tens of thousands of files. LWN.net +3↗ One developer reported Sublime Text consuming excessive file descriptors through inotify, forcing migration to alternative approaches. Fanotify improves scalability by monitoring entire mount points but requires CAP_SYS_ADMIN privileges and struggles with thousands of containers each in separate mount namespaces. Elastic +3↗ The audit subsystem captures user and process information but imposes **non-negligible performance penalties** and uses inefficient string-based socket communication between kernel and userspace. Elastic↗

eBPF eliminates these bottlenecks through **pre-filtering events in kernel space**, reducing transfer overhead by only forwarding relevant events. Aqua Security↗ After initial attachment, eBPF programs require only CAP_BPF and CAP_PERFMON rather than persistent elevated privileges. Theshoemaker +2↗ Cross-container attack research measured specific overhead differences: CapBits using eBPF incurs **less than 5% CPU overhead and less than 5% latency overhead**, while LSM/LSM-bpf approaches impose **greater than 31% CPU overhead and 16% latency overhead**. USENIX↗ The ZERO monitoring framework achieves **0.1% CPU utilization in control plane and 0% in data plane**, with 1-2 orders of magnitude latency reduction versus baselines like Netdata for Redis monitoring. eunomia↗USENIX↗

Real-world deployments demonstrate eBPF's feasibility at scale. The BMC memcached acceleration study from USENIX NSDI 2021 showed **2.1 microsecond median latency** for eBPF-accelerated cache hits versus 21.8 microseconds for standard memcached, delivering **18x throughput improvement** with negligible overhead. eunomia↗ USENIX↗ FSProbe sustains **15,000+ events per second** over 10-second periods with extensive benchmarking across different dentry resolution methods. GitHub↗github↗ Production deployments at Netflix and Facebook run BCC tools on all servers by default, with overhead low enough for continuous operation capturing CPU sampling, memory allocation, I/O patterns, and network flows simultaneously. Brendan Gregg↗Brendan Gregg↗

Machine learning achieves 99% accuracy predicting file access

Artificial intelligence and machine learning integration with filesystem monitoring represents the frontier of intelligent storage systems. Academic research from Huawei and Cyprus University demonstrated **streaming machine learning models achieving 99.5% accuracy predicting next file offsets** and 99.2% precision classifying file "hotness" based on future access rates. MDPI↗ These models employ Adaptive Random Forest for regression and Hoeffding Trees for classification, processing 14.5 million training instances in real-time with only 0.027 mean absolute error. The streaming approach delivers **62.5% reduction in runtime** versus batch-based retraining while using dramatically less memory by avoiding historical dataset storage. mdpi↗MDPI↗

The critical insight lies in multi-dimensional feature engineering combining temporal, spatial, contextual, semantic, and historical signals. MDPI↗ Top-ranked features by SHAP analysis include current file hotness (0.89 importance weight), file fully-read frequency (0.85), request offset patterns (0.82), file size (0.76), time since creation (0.71), and directory access frequency (0.68). Temporal features capture access frequency over sliding windows of 1 hour, 1 day, 1 week, and 1 month with exponential decay weights. Spatial features detect sequential versus random access patterns and locality within directory structures. MDPI↗ Contextual signals incorporate user identity, device type, application requesting access, and sharing status. Semantic understanding comes from file type, size, growth rate, and related file access patterns. mdpi↗

Dropbox's ML-powered file suggestion system serving 700+ million users illustrates production-scale deployment. The company evolved from simple heuristic-based "frecency" scores (combining frequency and recency) to sophisticated neural networks with learned embeddings. Dropbox↗ User embeddings capture individual behavior patterns while file embeddings represent semantic relationships. The learning-to-rank formulation optimizes hit ratio—the probability that suggested files will be accessed in the next hour. Dropbox↗ Neural networks enable non-linear feature combinations that capture complex interactions between recency, frequency, collaboration patterns, and temporal context like time of day and device type. dropbox↗ The system delivers measurable CTR improvements across all user segments while handling billions of files.

Predictive prefetching research demonstrates practical benefits from ML-driven filesystem intelligence. Springer↗ Springer↗ Google's Thesios analysis of distributed storage I/O traces showed **ML models achieving 93% F1 score** predicting next access versus 59% for rule-based approaches. ResearchGate↗ DeepPrefetcher's deep learning framework for SSD prefetching reduces read latency by 20-40% through pattern recognition at block level. NCache's ML-based cache management achieves **30% improvement over LRU/LFU** algorithms using features including access frequency, recency, file size, and access pattern type. mdpi↗ The Baleen flash cache system from CMU and Meta delivers **90% improvement in deep learning epoch time** by learning which data to cache and when to prefetch based on observed patterns. dropbox↗

AI-driven backup systems achieve 20-40% cost reduction

Intelligent backup systems represent the most mature commercial application of ML to filesystem operations, though few leverage eBPF for data collection. Veeam↗Connected↗ **Veeam's AI-enhanced platform** uses predictive analytics to forecast hardware failures and optimize storage management, automated scheduling that adjusts backup times based on data activity patterns, intelligent restoration prioritizing critical data recovery, and anomaly detection identifying ransomware before attacks fully materialize. Veeam↗ Enterprise deployments report **15-20% cost reduction** through predictive capacity planning and optimized resource allocation. veeam↗ A healthcare case study documented 25% storage reduction through intelligent deduplication and scheduling, while Contextual AI achieved 38% per-TB cloud cost reduction.

Alcion, recently acquired by Veeam, pioneered intelligent backup scheduling using ML to analyze user activity patterns and determine optimal backup times. Rather than fixed schedules, the system backs up during peak activity periods to ensure maximum data currency, adapting dynamically as work patterns evolve. Unitrends↗ The approach leverages time zone awareness and non-invasive monitoring of data activity peaks without requiring application instrumentation. Alcion↗ Unitrends employs ML for ransomware pattern detection in backup data, predictive recovery suggesting optimal file versions to restore, and proactive recovery initiating before users detect outages. Unitrends↗ Their recovery assurance capability uses automated testing to validate backup integrity continuously. Unitrends↗

Data criticality scoring forms the foundation of prioritization mechanisms. ML models classify importance based on access frequency and recency using temporal decay functions, user role and data ownership from access control metadata, file type and business function mapping applications to criticality levels, and compliance requirements from regulatory frameworks. Public Spectrum↗ Dynamic scheduling adjusts backup frequency per file or directory based on observed change rates, increases frequency during detected peak activity periods, and reduces overhead for rarely accessed data. Incremental forever strategies combine an initial full backup with continuous increments, using ML to determine when synthesizing new full backups proves more efficient than maintaining long delta chains. NAKIVO↗ NovaBACKUP↗ Case studies document **25-40% storage reduction** from intelligent increment management.

Predictive capacity management forecasts storage needs 1-4 weeks in advance with high accuracy. Datadog↗ Pure Storage's Pure1 platform uses ML to predict capacity exhaustion, recommends optimal expansion amounts, and provides customizable scenario modeling. ManageEngine OpStor ingests time-series storage data for trend prediction with graph-based visualizations and confidence intervals. Medium↗Pure Storage↗ NetApp's File System Analytics provides real-time visibility into usage trends, volume-level insights identifying hot spots, and active versus dormant data classification. NetApp↗ These systems alert proactively before capacity issues impact operations, enabling planned expansions rather than emergency responses. Datadog↗ Organizations report achieving **99%+ uptime** through predictive interventions.

Ransomware detection combines eBPF monitoring with ML classification

The convergence of eBPF filesystem monitoring and machine learning proves most mature in security applications. Research from 2024 demonstrates a **two-phase ransomware detection approach achieving 99.76% accuracy**. Phase one uses eBPF tracepoints on exec syscalls for hash-based detection of known threats. Phase two monitors file creation events via eBPF while ML classifies ransom notes in real-time using TF-IDF features and Multinomial Naive Bayes with Chi-Squared feature selection. arXiv↗ The system detects distinctive ransomware behavioral patterns including abnormally high file creation rates and rapid sequential modifications across directory trees. Zero-day attacks with no prior signatures are caught within seconds through behavioral analysis with minimal overhead running entirely at kernel level. eunomia↗arXiv↗

Flow-based intrusion detection from CN-TU Vienna implements **decision trees running entirely in eBPF** for per-packet classification considering full flow context. Network flow statistics including packet counts, bytes transferred, duration, and TCP flags feed the in-kernel model. Performance measurements show **20%+ faster execution** than equivalent userspace implementations by eliminating kernel-userspace context switches. ResearchGate↗arXiv↗ SmartX's BiLSTM model with eBPF/XDP achieves line-rate packet processing at 10Gbps+, dropping malicious packets before reaching userspace. eunomia↗ The architecture demonstrates eBPF's capability to host simple ML models directly in kernel for ultra-low-latency decisions while complex models run in userspace consuming eBPF data streams.

GPU workload monitoring represents an emerging application domain. The eACGM system merges eBPF kernel events with NVML GPU metrics for end-to-end performance analysis of training workloads, enabling fault diagnosis across CPU and GPU boundaries. USENIX↗ eInfer provides transparent tracing for distributed LLM inference, correlating per-request performance across CPU/GPU nodes with **less than 3% overhead**. AgentSight traces LLM agent activity without code changes by intercepting TLS traffic and kernel signals via eBPF, mapping high-level prompts to low-level system effects. eunomia↗Eunomia↗ These tools address observability challenges in AI workloads where traditional monitoring provides insufficient visibility into accelerator interactions.

Practical deployment architectures layer eBPF sensor networks, streaming data pipelines, ML model serving, and decision engines with feedback loops. eBPF probes collect syscalls, I/O operations, and network events. Feature extraction performs real-time aggregation over time windows, pattern detection for sequential/random/periodic access, statistical summaries, and context enrichment with user and application metadata. ML models employ streaming learning for online adaptation, batch models retrained periodically for complex patterns, and ensemble methods combining multiple signals. Decision engines enforce threshold-based rules, priority queues for actions, and closed-loop feedback for continuous improvement. Actions include cache placement, backup scheduling, data migration between tiers, prefetch issuance, and alert generation. mdpi↗

Implementation requires careful tradeoff balancing

Production deployment demands navigating multiple architectural decisions. Choosing between in-kernel versus userspace ML processing represents the fundamental tradeoff. Simple decision trees with limited depth can execute entirely in eBPF for **sub-microsecond inference latency**, suitable for real-time cache admission decisions or basic threat classification. DEV Community↗eBPF↗ Complex models including neural networks, autoencoders for anomaly detection, and RNN/LSTM sequence predictors require userspace or GPU execution, with eBPF serving as the high-performance data collection layer. ACM Digital Library↗ The pattern sees eBPF in kernel providing fast, low-overhead feature extraction while streaming aggregated data to userspace where sophisticated ML models operate.

Streaming versus batch machine learning presents another critical choice. Streaming ML using frameworks like River, MOA, or Vowpal Wabbit provides continuous adaptation to changing patterns, eliminates large historical dataset storage, delivers lower latency predictions, and handles concept drift naturally. MDPI↗ Batch ML requires periodic retraining but supports more complex models benefiting from GPU acceleration. The Huawei research demonstrated streaming ML achieving **99.5% F1 score with 0.021 milliseconds per prediction** versus batch ML requiring 231 seconds for retraining every 1 million records with 350MB storage overhead. mdpi↗MDPI↗ For dynamic workloads and resource-constrained environments, streaming approaches prove superior. Stable patterns with access to GPU resources may justify batch training.

Event filtering strategy dramatically impacts performance. Multi-level filtering starting with inode-based checks (fastest), then path prefix matching, operation type filtering, and per-file rate limiting reduces event volume by orders of magnitude. GitHub↗ Implementations should filter aggressively in kernel rather than forwarding everything to userspace. FSProbe's benchmarking showed the perf_buffer method most memory-efficient, sustaining higher event rates by caching resolved paths in userspace and maintaining compact kernel state. GitHub↗github↗ Per-CPU maps avoid lock contention for hot-path temporary storage while hash maps provide persistent cross-CPU state when necessary. Ring buffers deliver better ordering guarantees than perf buffers for applications requiring causality preservation.

Path resolution presents ongoing challenges due to kernel ABI instability. Functions can be renamed, inlined, or removed between versions. The bpf_d_path helper provides the cleanest solution but isn't available at all hook points. Manual dentry tree walking requires careful implementation respecting eBPF verifier constraints on loop bounds and memory access patterns. theshoemaker↗ BTF (BPF Type Format) and CO-RE (Compile Once - Run Everywhere) technologies partially address portability through runtime struct layout discovery and automatic relocation, but implementations must still handle variations. Eunomia↗ Production systems implement kernel version detection with appropriate fallbacks,

prioritizing fentry/fexit on modern kernels, LSM hooks for security monitoring, tracepoints for stable syscall-level coverage, and kprobes only as last resort.

Conclusion: intelligent storage systems are production-ready today

The convergence of eBPF's high-fidelity observability, machine learning's pattern recognition capabilities, and proven architectural patterns from industry leaders provides a compelling pathway to intelligent storage systems. Organizations can deploy incremental improvements immediately using available open-source tools and frameworks, gaining **20-40% efficiency improvements** through intelligent backup prioritization, predictive capacity planning, and ML-driven prefetching. The technology foundation exists in production at scale, with Netflix, Google, Dropbox, and major cloud providers demonstrating feasibility. Brendan Gregg +3↗

The notable gap in commercial eBPF-based backup solutions represents a clear market opportunity. While security vendors have embraced eBPF extensively, traditional backup vendors remain focused on AI-driven scheduling without leveraging kernel-level change monitoring. Combining eBPF's real-time filesystem telemetry with streaming machine learning for relevance scoring and prioritization could fundamentally disrupt backup architectures relying on periodic metadata scanning. Such systems would detect changes at sub-second latency with minimal overhead, prioritize backups based on multi-dimensional importance metrics, and adapt continuously to evolving access patterns.

For practitioners seeking to implement eBPF filesystem monitoring, the recommended approach prioritizes fentry/fexit hooks on VFS functions for modern kernels (5.5+), falling back to LSM hooks for security monitoring and tracepoints for broader compatibility. eBPF Docs +3↗ Production systems should implement aggressive in-kernel filtering, use ring buffers for event delivery, employ per-CPU maps to avoid contention, and leverage BTF/CO-RE for portability. Integration with ML systems follows established patterns: eBPF extracts features in kernel with sub-microsecond overhead, streaming pipelines aggregate data, and userspace models (streaming ML preferred) make decisions feeding back into system policies.

The future points toward self-optimizing storage systems with continuous eBPF monitoring feeding ML-driven automatic tuning, semantic understanding of file relationships and importance, and federated learning across distributed systems. Eunomia↗Medium↗ While fully autonomous systems remain 3-5 years out, the path forward is clear with each generation building on proven foundations. The combination of production-ready open-source tools (FSProbe, Tetragon, BCC, bpftrace), mature commercial platforms (Sysdig, Falco, Aqua), comprehensive academic validation, and demonstrated ROI positions eBPF as the foundation for next-generation intelligent storage infrastructure. amandeep singh +3↗